

SE116 - INTRODUCTION TO PROGRAMMING II

Term Project Report - 2025-2026

ObjectVille

A Zero-Player City Simulation

An object-oriented Java implementation of a tick-based city where zones, utilities, services and resources interact deterministically on a two-dimensional grid.

TEAM ASTEROX

Name	Student ID	Section
Yusuf Emir Yılmaz	20230602078	Section 3
Mehmet İnci	20230602036	Section 3
Mustafa Şen	20230602062	Section 1
Mehmet Kerem Saman	20230602058	Section 3
Umut Berk Şanlı	20230602060	Section 7

Table of Contents

1. 1. ProjectOverview	3
2. 2. Architecture and Package Structure	3
3. 3. Simulation Logic	4
4. 4. Q1 - Use of Inheritance, Polymorphism, Abstract Classes and Interfaces	5
5. 5. Q2 - Openness to Extension	6
6. 6. Q3 - Memory-Resident Structures and Generic Types	7
7. 7. Q4 - Lambda Expressions and Packaged Units	8
8. 8. Q5 - How the Map File Is Turned Into the City	8
9. 9. Q6 - Postmortem	9
10. 10. References	10

1. Project Overview

ObjectVille is a deterministic, zero-player simulation of a growing city laid out on a two-dimensional grid. The program reads a text map describing the placement of every cell, then advances the city through a fixed number of discrete time steps called ticks. On each tick the city distributes municipal services, pipes utilities outward from their providers, shares produced resources among consumers, recomputes the development level of every zone, and finally lets each zone produce its output. No user interaction occurs once the simulation starts: the entire evolution is fixed by the initial map and the rules encoded in the classes.

The project is written entirely in Java using object-oriented principles. It is split into clear packages, relies on abstract base classes and interfaces to model shared behaviour, and uses polymorphism so the engine can treat every cell uniformly while each cell type supplies its own rules. This report answers the six required questions in sections 4 through 9, after a short description of the model and the simulation loop.

This report answers the six required questions in sections 4 through 9, after a short description of the model and the simulation loop.

Map Symbols

Symbol	Cell	Role
H	Housing	Residential zone - produces population.
I	Industrial	Industrial zone - produces goods.
C	Commercial	Commercial zone - produces lifestyle.
R	Road	Connectable conduit utilities travel through.
P	PowerPlant	Utility provider - electricity (capacity 100).
W	WaterPumpingStation	Utility provider - water (capacity 100).
T	InternetHub	Utility provider - internet (capacity 100).
D	Hospital	Service provider - health, radius 3.
S	School	Service provider - education, radius 4.
F	PoliceStation	Service provider - security, radius 5.
E / space	EmptyCell	Empty, non-connectable terrain.

2. Architecture and Package Structure

The code lives under the `objectville` package, split into five sub-packages by responsibility. This separation keeps the simulation rules, the domain model and the input/output concerns independent of one another.

Package	Contents	Responsibility
cells	Cell, Zone, Housing, Industrial, Commercial, Road, EmptyCell, UtilityProvider (PowerPlant, WaterPumpingStation, InternetHub), ServiceProvider (Hospital, School, PoliceStation)	The domain model - every kind of tile on the map and its behaviour.
enums	ResourceType, ServiceType, UtilityType	Fixed type-safe categories used across the model.
interfaces	Connectable, ResourceConsumer, ResourceProducer	Capability contracts a cell may implement.
io	MapLoader, OutputWriter	Reading the map file and emitting the exact required output text.
simulation	Simulator, City, Position, ResourcePool	The engine: tick loop, grid storage, coordinates, shared resource totals.

Entry point. `Main` validates the command-line arguments (map path and tick count), asks `MapLoader` to build a `City`, wires it to an `OutputWriter`, and runs the `Simulator` for the requested number of ticks. Every error case - missing arguments, a non-numeric or negative tick count, an unreadable file, or an invalid map - is reported with a clear message rather than crashing.

3. Simulation Logic

Every tick runs the same fixed sequence of phases. Keeping the order constant is what makes the simulation deterministic and reproducible.

1 Reset	Each zone clears the utilities and services it received last tick.
2 Services	Every service provider grants its service to zones within its radius (straight-line distance).
3 Utilities	Each utility provider pushes its supply outward through the connectable network, in the order Internet, Water, Electricity.
4 Resources	From the second tick onward, last tick's produced resources are divided evenly among their consumers.
5 Update levels	Each zone recomputes its development level from the conditions it now meets.
6 Production	Each zone produces its output for this tick; the totals feed the next tick's resource phase.

Utility distribution (BFS). Each provider has a capacity of 100 units. Supply spreads outward from the provider through orthogonally adjacent connectable cells using a breadth-first search, so cells are reached in increasing number of hops. Within a single BFS layer the cells are served in a fully deterministic order - first by Manhattan distance to the provider, then by column, then by row - and each zone takes the smaller of its still-unmet demand and the provider's remaining capacity. A zone may be topped up by more than one provider in the same tick, and distribution stops as soon as the 100 units are exhausted.

Resource sharing. Population, goods and lifestyle produced in one tick are pooled and split by integer division among their consumers in the next tick: housing consumes lifestyle, industrial consumes population, and commercial consumes both population and goods. Any remainder simply stays unused.

Zone levels (0-3)

Zone	Level 1	Level 2	Level 3
Housing	Has electricity, water & internet	+ all three services	+ receives lifestyle
Industrial	Has electricity & water	+ security	+ receives population
Commercial	Has electricity, water & internet	+ security	+ receives population & goods

Losing a required utility drops a zone straight to level 0; otherwise a zone may rise or fall by at most one level per tick.

4. Q1 - Use of Inheritance, Polymorphism, Abstract Classes and Interfaces

This section answers where each object-oriented mechanism is used and, more importantly, why it was the right tool there.

Inheritance - where

The cell hierarchy is rooted at the abstract class `Cell`, which stores a cell's `Position`. `Zone` extends `Cell` and adds the state common to every zone - its level, the utilities it has received and the services it has received. The three concrete zones `Housing`, `Industrial` and `Commercial` extend `Zone`. In parallel, `UtilityProvider` and `ServiceProvider` are abstract subclasses of `Cell` whose concrete kinds are `PowerPlant` / `WaterPumpingStation` / `InternetHub` and `Hospital` / `School` / `PoliceStation`.

Inheritance - why

Every tile shares a position and the fact that it occupies one grid square, so that state belongs once on `Cell` rather than being duplicated in every class. The three zones share a level, a utilities map and a services map, so those live on `Zone`. Inheritance lets the shared data and the shared bookkeeping (resetting per-tick inputs, recording received utilities) be written a single time and reused, which is exactly what keeps the subclasses small and focused on what is genuinely different about them.

Abstract classes - where and why

`Cell`, `Zone`, `UtilityProvider` and `ServiceProvider` are all abstract. They are abstract because a bare "cell" or a bare "zone" is not a real tile you can place on the map - only the concrete kinds are. `Zone` declares abstract methods such as `recomputeLevel()`, `produce(...)` and `getUtilityDemand(...)` that it cannot implement itself because the answer differs per zone, yet it can still provide the shared framework around them. Marking the class abstract makes that contract explicit and stops anyone from instantiating an incomplete tile.

Interfaces - where and why

Three small interfaces describe capabilities that cut across the class hierarchy. `Connectable` is a marker interface implemented by every tile that utilities may flow through (roads, zones and utility providers); the BFS simply tests cell instance of `Connectable` to decide whether to spread further. `ResourceProducer` declares `produce(...)` and `ResourceConsumer` declares `demand(...)` and `receive(...)`. Interfaces are the right choice here because "can be flowed through" or "can consume a resource" are abilities that do not line up with a single branch of the inheritance tree, and a class implements only the contracts it actually needs.

Polymorphism - where and why

The engine holds cells as Cell / Zone references and, in its hot path, never asks what concrete kind it has. When it calls `zone.recomputeLevel()` or `zone.produce(resource)`, the JVM dispatches to the right subclass override at run time. This is why a new zone type can be dropped in without touching the loop. `instanceof` is used only where the rule genuinely depends on the category - for example deciding that housing consumes lifestyle while commercial consumes population and goods - and even there it selects behaviour rather than reimplementing it.

5. Q2 - Openness to Extension

Because behaviour is reached through abstract methods and interfaces, most new requirements are added by writing a new class rather than by editing the engine. The two requested extensions are walked through below.

(a) A new "Governmental" zone

Suppose a governmental zone is added that, like the others, requires population to grow, but at a new level 4 produces a brand-new resource called law. The changes are localised:

What changes	How
enums	Add LAW to <code>ResourceType</code> so the new resource is a first-class, type-safe category.
A new class <code>Governmental</code>	Extends <code>Zone</code> (or implements the producer/consumer interfaces). Its <code>recomputeLevel()</code> encodes the level rules including the new level 4, <code>getUtilityDemand(...)</code> states its utility needs, and <code>produce(LAW)</code> returns its output.
<code>MapLoader.createCellFromSymbol</code>	Add one switch case mapping a new map symbol (e.g. G) to <code>Governmental</code> so it can appear on a map.
<code>OutputWriter.zoneDisplayName</code>	Add the display name used in the output lines for the new zone.
Simulator resource phase	Population is already distributed to consumers, so the new zone slots into the existing consumer handling; only the LAW production needs to be pooled like the other produced resources.

The base hierarchy, the BFS, the level-stepping rule and the service phase all stay untouched. The one place that needs slightly more thought is the level cap: the existing zones clamp the level to 0-3, so supporting level 4 means the clampbound is parameterised per zone rather than hard-coded - a small, well-contained change.

(b) A new Parks service

A parks service that provides a new recreational benefit is even simpler, because services are distributed by a single generic radius rule:

What changes	How
enums	Add RECREATIONAL to ServiceType.
A new class Park	Extends ServiceProvider, supplying its radius and returning RECREATIONAL as its service type. No new distribution code is needed - the existing service phase already serves every provider within its Euclidean radius.
Zone subclasses	Whichever zones benefit from recreation declare that they use the RECREATIONAL service (the same usesService mechanism already drives which services count toward a zone's level).
MapLoader / OutputWriter	One new symbol case and one display name, exactly as for any new tile.

In both extensions the simulation engine - the tick loop and the distribution algorithms - never changes. This is the practical payoff of the open/closed principle: the system is open to new zones and services but closed against edits to the code that already works.

The one place still touched by every new tile is the symbol switch in MapLoader. A natural next refactoring is to replace that switch with a registry - a Map with Character keys and Function values that maps each symbol to a constructor - so that registering a new tile becomes a single map entry and the loader itself never changes again. We kept the explicit switch for now because it is the clearest form to read and grade, but the registry is the obvious evolution once the set of tiles grows.

6. Q3 - Memory-Resident Structures and Generic Types

All state lives in memory in standard Java collections and as small sets of arrays. The generic type parameters make these structures type-safe, so the compiler rejects, for instance, putting a `ServiceType` key into a utilities map.

Structure	Generic type	Purpose
Grid	<code>Cell[][]</code>	The map itself - a 2-D array of tiles indexed by row and column.
Zone list	List of Zone (ArrayList)	Iteration order for deterministic per-tick updates and production.
Provider lists	List of UtilityProvider, List of ServiceProvider	The providers that drive the utility and service phases.
Utilities received	Map of UtilityType to Integer (HashMap)	Per-zone tally of how much of each utility arrived this tick.
Services received	Map of ServiceType to Boolean	Per-zone record of which services reached the zone this tick.
BFS visited set	Set of Position (HashSet)	Stops the breadth-first search from revisiting cells.
BFS frontier	List of Position (ArrayList)	The current and next BFS layers being expanded.
Resource pool	EnumMap of ResourceType to Integer	Accumulates produced resources for sharing next tick.

The choice of structure follows the access pattern. A `Cell[][]` gives $O(1)$ lookup by coordinate, which the BFS needs constantly. A `HashSet` of `Position` gives $O(1)$ membership tests for the visited check, which relies on `Position` correctly overriding `equals` and `hashCode`. The per-zone Maps are keyed by the relevant enum so a lookup is both fast and impossible to mis-key. The resource pool deliberately uses an `EnumMap` rather than a `HashMap`: because the keys are a small fixed enum (`ResourceType`), an `EnumMap` is backed by a compact array, gives faster lookups and iterates in the enum's natural order, which keeps resource sharing deterministic.

7. Q4 - Lambda Expressions and Packaged Units

Packaged units. The code is organised into the five packages listed in section 2 (objectville.cells,

.enums, .interfaces, .io and .simulation), each a compilation unit grouping classes by responsibility. Packaging keeps the domain model separate from the engine and from input/output, controls visibility, and makes the dependency direction obvious - the engine depends on the model, not the other way around.

Lambda expressions. Lambdas are used where a short piece of behaviour is passed to a library method. The clearest examples are in the simulator:

```
// reset every zone's per-tick inputs (Consumer of Zone)
city.getZones().forEach(zone -> zone.resetTickInputs());

// order one BFS layer deterministically (Comparator of Position)
frontier.sort((a, b) -> {
    int da = a.manhattanDistance(start);
    int db = b.manhattanDistance(start);
    if (da != db) return da - db;
    if (a.getCol() != b.getCol()) return a.getCol() - b.getCol();
    return a.getRow() - b.getRow();
});
```

The first lambda is a Consumer of Zone handed to forEach; the second is a Comparator of Position handed to List.sort. Both let a small, single-use behaviour be expressed inline instead of as a separately named class, which keeps the tie-breaking rule for utility distribution right next to where it is used.

8. Q5 - How the Map File Is Turned Into the City

MapLoader.load(String filePath) reads the map and builds the in-memory grid. The steps are:

Read	A BufferedReader over a FileReader reads the file line by line into a list. Trailing blank lines are dropped so they do not become phantom rows.
Detect dimensions	If the first line matches the regular expression for two integers separated by whitespace, it is treated as an explicit "rows cols" header. Otherwise the loader is header-less and infers the number of rows and columns from the grid itself.
Normalise	Each grid line has its spaces and tabs stripped so that alignment whitespace in the file does not shift the columns.
Build cells	For every character, createCellFromSymbol(symbol, position) returns the matching Cell subclass via a switch, and the cell is placed into the City at its Position.
Categorise	City.setCell inspects each added cell with instanceof and files it into the right list - zones, utility providers or service providers - so later phases can iterate just the cells they need.

The symbol-to-class mapping is the switch in createCellFromSymbol: H maps to Housing, C to Commercial, I to Industrial, R to Road, P to PowerPlant, W to WaterPumpingStation, T to InternetHub, D to Hospital, F to PoliceStation, S to School, and E or any unrecognised character to EmptyCell. Because reading the map is isolated in this one class, supporting a new tile is a one-line addition here (as shown in Q2) and nothing else in the loader changes.

This is the project's main use of Java I/O: file reading with the java.io reader classes, wrapped in error handling so a missing or malformed file produces a clear message instead of an exception trace.

9. Q6 - Postmortem

What went well

Designing around abstract base classes and interfaces from the start paid off: once the tick loop was written against Zone and the three capability interfaces, individual zone and provider rules could be implemented and tested in isolation. Splitting the code into packages kept responsibilities clear and made it easy for several team members to work in parallel. Driving every category through enums (ResourceType, ServiceType, UtilityType) removed a whole class of typo bugs.

What was hard

The single hardest problem was making the utility-distribution output match the reference exactly. Breadth-first search visits cells in the right number of hops, but when two providers can reach the same cells, the order in which cells inside one layer are served changes which provider fills which zone, and therefore changes the printed lines. We tightened the within-layer ordering to a deterministic key - Manhattan distance to the provider, then column, then row - which aligned the great majority of the output with the reference. A handful of lines on one tick come from an irreducible tie between two internet hubs that compete for the same boundary cells at the moment one runs out of capacity; resolving those would require a rule the specification does not pin down, so we documented the behaviour rather than over-fitting to one sample.

Common pitfalls we avoided

Because HashSet of Position is central to the BFS, Position carefully overrides both equals and hashCode - overriding only one is a classic bug that would have let the search revisit cells or loop. Per-tick state is reset at the top of every tick so values never leak between ticks. Resource sharing uses integer division deliberately and discards the remainder, matching the specification rather than silently accumulating fractional amounts. Inputs are validated in Main so bad arguments fail with a message instead of an exception.

What we would do next

With more time we would add automated regression tests that diff the program's output against saved reference logs for several maps, so any future change is checked instantly, and we would make the level-cap configurable per zone up front to make additions like the governmental zone in Q2 completely edit-free.

10. References

Oracle. The Java Tutorials - Interfaces and Inheritance. Oracle Corporation.
<https://docs.oracle.com/javase/tutorial/java/landl/>

Oracle. The Java Tutorials - Lambda Expressions. Oracle Corporation.
<https://docs.oracle.com/javase/tutorial/java/javaOO/lambdaexpressions.html>

Oracle. Java Platform, Standard Edition API Specification - java.util Collections Framework and java.io. Oracle Corporation.

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley.

Bloch, J. (2018). Effective Java (3rd ed.). Addison-Wesley. (Item 10-11 on the equals/hashCode contract; Item 20-21 on interfaces.)

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). Introduction to Algorithms (3rd ed.). MIT Press. (Breadth-first search.)